SIMATS ENGINEERING
ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0515 Slot B

COURSE OUTCOME COVERED
CO5: ***Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos.*** **(BTL 6)**

Assessment Tool Weightage: 10%

QUESTIONS:4 Total Marks 20 Duration :60 Minutes Annexure A
Mock Interview

Code of Conduct:

I R Dinesh-192421117 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: R Dinesh

Rubrics for Evaluation

Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	***Marks(100)***
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge	
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem solving ability; requires guidance	Unable to solve or apply concepts	
Analytical Thinking	15	Analyzes questions critically and provides well structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated	
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively	

Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism	
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately	

OpenCV Basics & Pipeline Thinking

1. Design and explain a complete OpenCV pipeline to read, process, and display multiple images from a folder efficiently. Clearly justify your design choices and discuss potential challenges and optimizations.
2. A video file is not loading properly in OpenCV. Analyze the possible causes and propose a structured debugging approach, explaining each step logically.
3. You need to overlay text and shapes on a live video feed. Design your solution and explain how you would implement it efficiently, including key OpenCV functions and reasoning. 4. An image appears too dark when read. Analyze the issue and propose modifications to your processing pipeline, justifying how your approach improves visibility.

1. OpenCV Pipeline to Process Multiple Images from a Folder

Answer

I would design the system as a **batch image-processing pipeline**:

Folder → Image Acquisition → Validation → Preprocessing → Processing → Display/Save

Step 1: Image Acquisition

First, I would obtain all supported image files such as .jpg, .jpeg, and .png from the folder using Python's os or glob module.

Each image can be read using:

img = cv2.imread(filename)

Step 2: Validation

I would check whether the image was successfully loaded.

```
if img is None:
    continue
```

This prevents corrupted or unsupported files from stopping the complete process.

Step 3: Preprocessing

Depending on the application, I would resize large images using:

cv2.resize()

and convert them to grayscale when color information is not required:

cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

This reduces memory usage and computational complexity.

Step 4: Image Processing

The required operation can then be applied, such as:

- Noise removal → cv2.GaussianBlur()
- Edge detection → cv2.Canny()
- Thresholding → cv2.threshold()

Step 5: Display and Storage

The processed image can be displayed using:
cv2.imshow()
and saved using:
cv2.imwrite()

Optimization and Challenges

The main challenges are **different image sizes, corrupted files, large datasets, and memory usage**. To optimize the system, I would process **one image at a time** rather than loading all images into memory. I would also resize very large images and skip invalid files.

Final Pipeline

Read folder → Load image → Validate → Resize/Preprocess → Process → Display → Save → Next image

Conclusion

This pipeline is efficient because it processes images sequentially, reduces memory usage, handles invalid files safely, and can be easily extended with different OpenCV operations.

---

2. Video File Not Loading Properly in OpenCV

Answer

If a video is not loading, I would not immediately change the program. I would follow a **systematic debugging process**.

Step 1: Verify the File Path

First, I would check whether the video actually exists.

```
import os

print(os.path.exists("video.mp4"))
```

If it returns False, the path or filename is incorrect.

Step 2: Check Video Opening

I would use:

```
cap = cv2.VideoCapture("video.mp4")

if not cap.isOpened():
    print("Video cannot be opened")
```

If isOpened() returns False, OpenCV cannot access the video.

Step 3: Check Frame Reading

Next, I would try to read a frame:

```
ret, frame = cap.read()

print(ret)
```

If ret is False, the video may be corrupted or its codec may not be supported.

Step 4: Check Video Properties

I would check:

```
fps = cap.get(cv2.CAP_PROP_FPS)
width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
```

This confirms whether OpenCV is correctly reading the video information.

Step 5: Check Format and Codec

If the path and code are correct, I would check whether the video format or codec is supported. I would test a standard .mp4 video or convert the video to a compatible format.

Step 6: Test with a Simple Loop

```
while True:
```

```
    ret, frame = cap.read()

    if not ret:
        break

    cv2.imshow("Video", frame)

    if cv2.waitKey(30) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Possible Causes

1. Incorrect file path
2. Missing video file
3. Unsupported codec
4. Corrupted video
5. Incorrect VideoCapture() usage
6. OpenCV/FFmpeg issue
7. Permission problem

Conclusion

My debugging sequence would be:

File Path → VideoCapture() → isOpened() → read() → Video Properties → Format/Codec → OpenCV Installation

This approach identifies the actual cause systematically instead of modifying the program randomly.

---

3. Overlay Text and Shapes on a Live Video Feed

Answer

I would create a real-time pipeline:

Camera → Capture Frame → Draw Shapes/Text → Display → Repeat

Step 1: Capture Live Video

I would open the webcam using:

```
cap = cv2.VideoCapture(0)
```

Then continuously capture frames:

```
ret, frame = cap.read()
```

Step 2: Draw Shapes

For example, I can draw a rectangle using:

```
cv2.rectangle(frame, (50,50), (250,200), (0,255,0), 2)
```

A circle can be drawn using:

```
cv2.circle(frame, (300,200), 50, (255,0,0), 2)
```

A line can be drawn using:

```
cv2.line(frame, (0,300), (500,300), (0,0,255), 2)
```

Step 3: Add Text

I would use:

```
cv2.putText(
    frame,
    "Live Video",
    (50,40),
    cv2.FONT_HERSHEY_SIMPLEX,
```

```
    1,
    (255,255,255),
    2
)
```
This can be used to display information such as object names, FPS, timestamps, or system status.

Step 4: Display the Result

```
cv2.imshow("Live Video", frame)
```
The process continues for every frame.

Step 5: Exit and Release Resources

I would use:
```
if cv2.waitKey(1) & 0xFF == ord('q'):
    break
```
Then:
```
cap.release()
cv2.destroyAllWindows()
```
Efficiency

To maintain real-time performance:

- Process only the current frame.
- Avoid unnecessary image copies.
- Reduce frame resolution if the camera produces very large frames.
- Keep drawing operations lightweight.
- Use waitKey(1) for responsive display.

Final Pipeline

Webcam → Frame Capture → Rectangle/Circle/Line/Text → Display → Next Frame

Conclusion

OpenCV drawing functions are computationally lightweight, so text and shapes can be added to live video while maintaining real-time performance. This approach is useful for **object detection, surveillance, face detection, and traffic monitoring systems**.